LABORATORY WORK NO. 2

Cryptanalysis of Monoalphabetic Ciphers

Breaking an intercepted cryptogram by frequency analysis

Discipline: **Cryptography**

Author: **Mutu Adrian**

Group: **FAF-242**

Variant: **15**

16 October 2026

1. Short theory

A monoalphabetic cipher replaces every occurrence of a letter with the same substitute throughout the message. Its key is an arbitrary permutation of the alphabet, so the key space is $26! \approx 4.03 \times 10^{26}$ — far too large to search. The cipher is nevertheless broken routinely, because the key space is not where its weakness lies.

The weakness is that the substitution leaves the *structure* of the language untouched. Each plaintext letter maps to one ciphertext letter and always the same one, so everything that makes a letter recognisable survives into the ciphertext: how often it occurs, which letters it sits next to, which short words it forms, whether it doubles. The attacker does not need the key — the text carries enough of the language to give the key away.

Signal	What it gives away
Letter frequencies	`E` is the most common letter of English at 12.70%, then `T`, `A`, `O`, `I`, `N`
Short words	The commonest three-letter word is `THE`; two-letter words are mostly `OF`, `TO`, `IN`, `IS`, `AS`
Bigrams	`TH`, `HE`, `AN`, `IN`, `ER`, `ON`, `RE`, `ED`, `ND`, `HA`, `AT`, `EN`
Trigrams	`THE`, `AND`, `THA`, `ENT`, `ION`, `TIO`, `FOR`, `NDE`
Doubled letters	`SS`, `EE`, `TT`, `OO` and `FF` are the frequent ones

The attack works by hypothesis and confirmation rather than by calculation. The frequency order of the cryptogram is lined up against that of the language, which suggests a few substitutions; those are made, and the partly decrypted text is read to see which words are beginning to appear. A word that becomes readable fixes several more letters at once, and each new letter makes more words readable. The process converges quickly and is self-checking — a wrong guess produces unpronounceable text and is abandoned.

Two conditions have to hold: the message must be long enough for its letter frequencies to resemble those of the language, and the language of the plaintext must be known. Both are satisfied here.

2. The intercepted cryptogram

Variant 15. The message is 3906 characters long, of which 3165 are letters. Two things are known about it: it was enciphered with a monoalphabetic cipher, and the plaintext is English. Only the letters were enciphered — digits, punctuation, spaces and the capitalisation are exactly as they were in the original, which is a considerable help, because the word boundaries are visible.

```
WQV WVSVJITUQ ztov hifuwnjituqf rqtw xw xp wnotf. Ptzdvs C. A. Znipvpvgw "Rqtw qtwq Jno rindjqw!" xg
1844. Wqv gvyw fvti qxp strfvi tgouinznwxngts tjvgw, Citghxp N. E. Pzxwq, udasxpqvo t hnzzvihxts
hnovvgwxwsvo Wqv Pvhivw Hniivpungoxgj Knhtadstif; Totuwvo cni Dpv wn Znipv'pVsvhwin-Ztjgvwxh Wvsvjituq,
xg rqnpv uivcthv qv ovhstivo wqtw "pvhivhf xghniivpungovghv, xp cti w qv znpw xzuniwtgw hngpxovitwxng."
Wqxp rtpuinkxovo af t pduvi - vghxuqvizvgw.Tp wqv znpw vyhxwxgj xgkvgwxng nc wqv cxipw qtsc nc wqv
hvgwdif, wqvwvsvjituq pwxiivo tp zdhq xgwvivpw xg xwp otf tp Pudwgxl oxo xg xwp. Wqvjivtw tgo rxovsf
cvsw gvvo cni pvhivhf tr tlvgvo wqv stwvgw xgwvivpw xghxuqvip wqtw pn ztgf uvnusv pvvz wn qtkv, tgo
lxgosvo t gvr xgwvivpw xgztgf nwqvip. Onmvgp nc uvipngp twwvzuwvo wn oivtz du wqvxi nrgdgaivtltasv
hxuqvip. Wqvxi hngwixadwxngp vgixhqvo xw rxwq onmvgp nc gvrhxuqvi pfpwvzp.Tp adpxg vppzvg tgo wqv
udasxh
dpvo wqv wvsvjituq zniv tgo zniv,wqvf cndgo wqtw wqvxi cvtip tandw sthl nc uixkthf rviv vytjjvitwvo.
Wqvhsvilp ovtsw xzuvipngtssf rxwq wqv zvpptjvp. Wqv wvsvjituq hnzutgxvpivpuvhwvo wqvxi hngcxovgwxtsxwf.
Tgo hnzzvihxts hnovp sxlv Pzxwq'p, rqxhqivusthvo rniop tgo uqitpvp af pxgjsv hnovrniop ni hnov -gdzavip
wn hdw wvsvjituq wnssp, tccniovo pdccxhxvgw pvhdixwf cni znpwadpxgvpp witgpthwxngp af pxzusf uivhsdoxgj
tg tw-pxjqw hnzuivqvgpxng ncwqv zvtgxgj. Wqv ainlvip tgo witovip pnng ivtsxmvo wqtw wqv ztxgtoktgwtjv
```

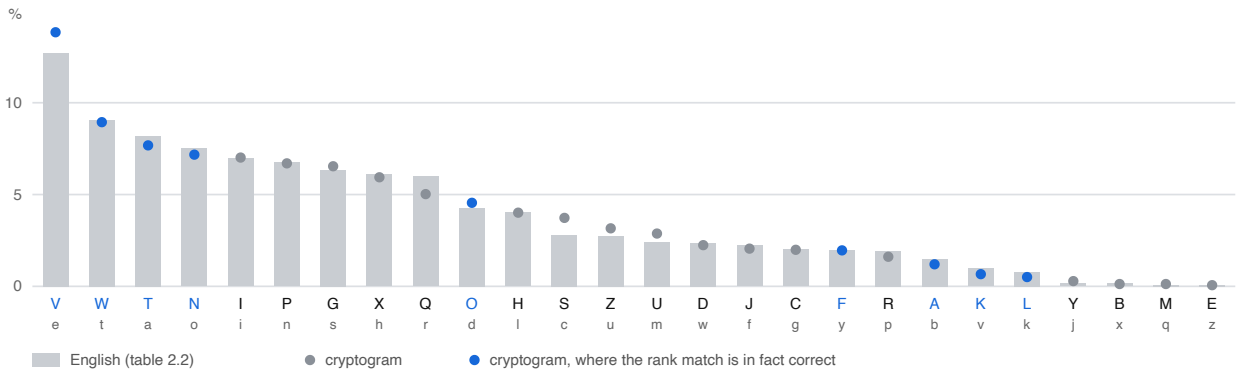
```
nc
wqvpv hnovp rtp wqvxi vhngnzf.Jnkvigzvgw zxgxpwixvp dpvo wqv wvsvjituq, wnn. Tw cxipw wqvf zdpwqtkv
vghnovo rxwq wqvxi gnzvghstwnip. Adw tswqndjq pvhivhf rtputitzndgw cni wqvz, wqvf sxlvo wqv wvsvjituqxh
vhngnzf nc t stijv hnov —vpuvhxtssf tp wqvf wvsvjituqvo zniv tgo zniv. Pn rqvg wqv wxzv tiixkvown
hnzuxsv t gvr gnzvghstwni, wqvf tatgongvo wqtw cniz, hnuxvo wqvhnzzvihxts cniz, tgo uinodhvo t cdss
—csvojvo hnov. Wqv gnzvghstwnipqto qto wqvxi 1,— ni 2,000 hnov—gdzavip xg zxyvo niovi , adw wqv rtitgo
cnivxjg zxgxpwixvp atslvo tw wqv vyuvgpv nc oitrxgj du t 50,000 —vgwif hnov xg wrn utiwp, tgo wqvf qto
gn uincvppxngts hifuwtgtsfpwp wnrtig wqvz nc wqv otgjvi nc wqv ngv —utiw cniztw. Wqvf ivsxvo cni
pvhdixwfdung pztss voxwxngp, axj ptcvp, v ywvgpxkv svyxhng (stijv hnovp tiv qtioviwn aivtl wqtg pztss
ngvp, nwqvi wqxgjp avxgj vbdts), tgopduvivghxuqvizvgw, ivwtxgxgj hnovgdzavip wn cthxsxwtwv wqxp xgpwvto
ncprxwhqxgj wn hnovrniop. Wqxp vknsdwxng rtp vppvgwxtssf hnzusvwv af wqv1860p. Wqv stijv, ngv—utiw hnov
qto ivusthvo wqv pztss, wrn —utiwgnzvghstwni xg qxjq—svkvs zxsxwtif tgo oxusnztwxh
hifuwnjituqf.Zvtgrqxsv, wqv wvsvjituq, tdwqni nc wqxp ovkvsnuzvgw, rtp hivtwxgjpnzvwqxgj gvr xg
rti—pxjgts hnzzdgxhtwxngp, ni knsdzxgndp hnzztgotgo ivhnggtxpptghv zvpptjvp. Nc hndipv pdhq zvpptjvp
qto
vyxpwvoavcniv, rxwq wnihqvp, uxjvngp, tgo hndixvip, adw xg pn itivcxvo t cniz wqtwwqvf rviv gnw vkvg
htssvo "pxjgts hnzzdgxhtwxngp." Wqv wvsvjituqvgtasvo hnzztgovip, cni wqv cxipw wxzv xg qxpwnif, wn
vyviw
xgpwtgwtgvn dptgo hngwxgdndp hngwins nkvi jivtw ztppvp nc zvg puivto nkvi stijvtivtp.Wqvpv wthwxhts
zvpptjvp ivbdxivo uinwvhwxng: wvsvjituq rxivp hndso avwtuuvo. Gvxwqvi wqv nso gnzvghstwni gni wqv gvr
hnov rndso on. Wqvfrviv wnn vtpf wn htuwdiv xg hnzatw, wnn qtio wn ivxppdv bdxhlsf tgocivbdvgwsf wn wqv
gdzvindp tgo rxovpuivto wvsvjituq unpwp. Pxjgtsnccxhvip wdigvo trtf cinz wqvz. Wqvf snnlvo xgpwvto wn
wqtw gvjsvhwvohqxso nc hifuwnjituqf, wqv hxuqvi. Hxuqvip hndso av uixgwvo hqvtusf ng tpxgjsv pqvvw nc
utuvi tgo oxpw ixadwvo rxwq vtpv. Pvhivhf rtp atpvo dungktixtasv lvfp, pn htuwdiv nc wqv jvgvits pfpwvz
tgo vkvg nc ngv nc wqvlvfp rndso gnw hnzuinzxpv tss tg tizf'p pvhivw zvpptjvp. Pnsdwxngprndso av
uivkvgwvo af ituxo lvf hqtgjvp. Hxuqvip rviv xovts cni atwwsv —mngv zvpptjvp, tgo wqv cxipw nc wqv
znovig rtip, wqv Tzvixhtg Hxkxs Rti,dpvo wqvz cni edpw wqtw. Wqdp rtp anig t gvr jvgiv xg hifuwnjituqf:
wqvcxvso hxuqvi.
```

A few words are run together (`Znipvpvgw`, `wqvwvsvjituq`) and a few have a stray space inside them (`v ywvgpxkv`). These come from the way the text was laid out in the assignment, not from the cipher, and they do not interfere with the attack.

3. Frequency analysis

Counting the letters of the cryptogram and placing them in decreasing order gives a profile that follows English very closely in shape. `V` leads at 13.84% against 12.70% for `E`, and the curve falls away at the same rate.

Rank	Cipher	%	English	%	Rank	Cipher	%	English	%
1	V	13.84	E	12.70	14	U	2.88	M	2.41
2	W	8.94	T	9.06	15	D	2.24	W	2.36
3	T	7.68	A	8.17	16	J	2.05	F	2.23
4	N	7.17	O	7.51	17	C	1.99	G	2.01
5	I	7.01	I	6.97	18	F	1.96	Y	1.97
6	P	6.70	N	6.75	19	R	1.61	P	1.93
7	G	6.54	S	6.33	20	A	1.20	B	1.49
8	X	5.94	H	6.09	21	K	0.66	V	0.98
9	Q	5.02	R	5.99	22	L	0.51	K	0.77
10	O	4.55	D	4.25	23	Y	0.28	J	0.15
11	H	4.01	L	4.03	24	B	0.13	X	0.15
12	S	3.73	C	2.78	25	M	0.13	Q	0.09
13	Z	3.16	U	2.76	26	E	0.06	Z	0.07



Letter frequencies of the cryptogram against English, by rank. Bars: English (table 2.2). Dots: the cryptogram. Blue marks the ranks at which matching the two orders happens to give the correct letter — 9 of 26.

The chart shows why counting alone does not finish the job. Lining the two orders up rank for rank is correct for only 9 of the 26 letters — but it is correct for the first four, and those four are the ones worth having. `E`, `T`, `A` and `O` account for roughly 38% of English text, so fixing them makes a large part of the message readable, and the rest of the attack is driven by the words that then appear rather than by further counting.

The other signals

Word boundaries survived the encipherment, so short words and repeated groups are visible too:

Signal	Most frequent in the cryptogram
Three-letter words	`WQV` (35), `TGO` (16), `CNI` (8), `GVR` (5), `RTP` (5), `QTO` (4)
Two-letter words	`NC` (19), `XG` (12), `WN` (11), `AF` (5), `TP` (5), `PN` (4)
One-letter words	`T` (9), `P` (3)
Trigrams	`WQV` (68), `TGO` (25), `ITU` (18), `JIT` (17), `TUQ` (17), `VGW` (16)
Doubled letters	`PP` (13), `SS` (10), `ZZ` (7), `NN` (5), `VV` (4), `II` (4)

Two entries stand out before any substitution is made. `WQV` occurs 35 times as a word and 68 times as a trigram — no English word other than `THE` behaves like that. And `T` standing alone 9 times can only be `A` or `I`, since those are the only one-letter words in English.

4. Breaking the cipher

The attack proceeds exactly as in section 2.3 of the assignment. After every deduction the substitutions found so far are applied and the result is read back. Letters already recovered appear in **lowercase**; letters still unknown stay as the **UPPERCASE** ciphertext letter, so a half-broken word such as `tIe` shows at a glance what is missing.

The running key is written as 26 characters — the plaintext letter that each ciphertext letter `a` to `z` stands for, with a dot where nothing has been decided yet. Only the opening of the message is reproduced at each step; `python3 main.py walkthrough` prints all eight stages in full.

Step 1 — the giveaway word

The three-letter group WQV occurs 35 times - far more than any other word. The commonest three-letter word in English is THE, and V is also the most frequent letter of the cryptogram (13.84%, against 12.70% for E in English).

```
key: ..............h....et...

the teSeJITUh ZTOe HIFUtNJITUhF RhTt Xt XP tNOTF. PTZDeS C. A. ZNIPePeGt "RhTt hTth JNO RINDJht!" XG
1844. the GeYt FeTI hXP STRFeI TGOUINZNtXNGTS TJeGt, CITGHXP N. E. PZXth, UDASXPheO T HNZZeIHXTS
HNOeeGtXtSeO the PeHIet HNIIePUNGOXGJ KNHTADSTIF; TOTUteO CNI DPe tN ZNIPe'PeSeHtIN-ZTJGetXH teSeJITUh,
XG RhNPe UIeCTHe he OeHSTIeO thTt "PeHIeHF XGHNIIePUNGOeGHe, XP CTI t he ZNPt XZUNItTGt HNGPXOeITtXNG."
```

Step 2 — the two-letter words

Among two-letter words, NC appears 19 times, XG 12 and WN 11. With W already standing for T, WN must be TO. The other two are then OF and IN, which also agrees with N being the fourth most frequent letter and matching O.

```
key: ..f...n......o..h....eti..

the teSeJITUh ZTOe HIFUtoJITUhF RhTt it iP toOTF. PTZDeS f. A. ZoIPePent "RhTt hTth JoO RIoDJht!" in
1844. the neYt FeTI hiP STRFeI TnOUIoZotionTS TJent, fITnHiP o. E. PZith, UDASiPheO T HoZZeIHiTS
HoOeentitSeO the PeHIet HoIIePUonOinJ KoHTADSTIF; TOTUteO foI DPe to ZoIPe'PeSeHtIo-ZTJnetiH teSeJITUh,
in RhoPe UIefTHe he OeHSTIeO thTt "PeHIeHF inHoIIePUonOenHe, iP fTI t he ZoPt iZUoItTnt HonPiOeITtion."
```

Step 3 — AND

TGO is the second most frequent three-letter word (16 times) and now reads a-n-d: AND. T is the third most frequent letter of the cryptogram, which fits A.

```
key: ..f...n......od.h..a.eti..

the teSeJIaUh Zade HIFUtoJIaUhF Rhat it iP todaF. PaZDeS f. A. ZoIPePent "Rhat hath Jod RIoDJht!" in
1844. the neYt FeaI hiP SaRFeI andUIoZotionaS aJent, fIanHiP o. E. PZith, UDASiPhed a HoZZeIHiaS
HodeentitSed the PeHIet HoIIePUondinJ KoHaADSaIF; adaUted foI DPe to ZoIPe'PeSeHtIo-ZaJnetiH teSeJIaUh,
in RhoPe UIefaHe he deHSaIed that "PeHIeHF inHoIIePUondenHe, iP faI t he ZoPt iZUoItant HonPideIation."
```

Step 4 — the title

The title of the passage, WQV WVSVJITUQ, now reads THE TE-E--A-H. The subject of the text and the pattern of repeated letters make TELEGRAPH the only sensible reading.

```
key: ..f...n.rg...od.h.lapeti..

the telegraph Zade HrFptographF Rhat it iP todaF. PaZDel f. A. ZorPePent "Rhat hath god RroDght!" in
1844. the neYt Fear hiP laRFer andproZotional agent, franHiP o. E. PZith, pDAliPhed a HoZZerHial
Hodeentitled the PeHret HorrePponding KoHaADlarF; adapted for DPe to ZorPe'PeleHtro-ZagnetiH telegraph,
in RhoPe prefaHe he deHlared that "PeHreHF inHorrePpondenHe, iP far t he ZoPt iZportant HonPideration."
```

Step 5 — IS, AS and WAS

TP occurs 5 times and reads A-. It cannot be AT, since T is already W, so TP is AS and P is S. This also makes XP read IS, and RTP (5 times) read -AS, that is WAS, so R is W.

```
key: ..f...n.rg...odshwlapeti..

the telegraph Zade HrFptographF what it is todaF. saZDel f. A. Zorsesent "what hath god wroDght!" in
1844. the neYt Fear his lawFer andproZotional agent, franHis o. E. sZith, pDAlished a HoZZerHial
Hodeentitled the seHret Horresponding KoHaADlarF; adapted for Dse to Zorse'seleHtro-ZagnetiH telegraph,
in whose prefaHe he deHlared that "seHreHF inHorrespondenHe, is far t he Zost iZportant Honsideration."
```

Step 6 — CRYPTOGRAPHY

The long word HIFUWNJITUQF now reads -R-PTOGRAPH-. The text is about secret writing, so this is CRYPTOGRAPHY: H is C and F is Y.

```
key: ..f..yncrg...odshwlapeti..

the telegraph Zade cryptography what it is today. saZDel f. A. Zorsesent "what hath god wroDght!" in
1844. the neYt year his lawyer andproZotional agent, francis o. E. sZith, pDAlished a coZZercial
codeentitled the secret corresponding KocaADlary; adapted for Dse to Zorse'selectro-Zagnetic telegraph,
in whose preface he declared that "secrecy incorrespondence, is far t he Zost iZportant consideration."
```

Step 7 — MADE and SAMUEL

ZTOV reads -ADE, so Z is M (MADE). PTZDVS then reads SAM-EL, so D is U, giving the name SAMUEL - Samuel Morse, who fits the subject.

```
key: ..fu.yncrg...odshwlapeti.m

the telegraph made cryptography what it is today. samuel f. A. morsesent "what hath god wrought!" in
1844. the neYt year his lawyer andpromotional agent, francis o. E. smith, puAlished a commercial
codeentitled the secret corresponding KocaAulary; adapted for use to morse'selectro-magnetic telegraph,
in whose preface he declared that "secrecy incorrespondence, is far t he most important consideration."
```

Step 8 — the rare letters

Only the rare letters are left, and each is now fixed by a single word: AV = -E is BE, KNSDZXGNDP = -OLUMINOUS is VOLUMINOUS, VYUVGPV = E-PENSE is EXPENSE, MNGV = -ONE is ZONE, VBDTS = E-UAL is EQUAL, EDPW = -UST is JUST and LVFP = KEYS fixes L as K.

```
key: bqfujyncrgvkzodshwlapetixm

the telegraph made cryptography what it is today. samuel f. b. morsesent "what hath god wrought!" in
1844. the next year his lawyer andpromotional agent, francis o. j. smith, published a commercial
codeentitled the secret corresponding vocabulary; adapted for use to morse'selectro-magnetic telegraph,
in whose preface he declared that "secrecy incorrespondence, is far t he most important consideration."
```

All 26 letters are now accounted for and the message reads as plain English.

5. The key and the message

Putting the two alphabets side by side gives the key, in the form of table 2.5 of the assignment:

```
ciphertext   a b c d e f g h i j k l m n o p q r s t u v w x y z
plaintext    b q f u j y n c r g v k z o d s h w l a p e t i x m
```

Read in the other direction — what each plaintext letter was enciphered to — the key is:

```
plaintext    a b c d e f g h i j k l m n o p q r s t u v w x y z
ciphertext   t a h o v c j q x e l s z g n u b i p w d k r y f m
```

A check on the answer

The second line is not a random permutation. Writing each letter as its position in the alphabet, the encipherment is

$$c = (7x + 19) \bmod 26$$

an affine cipher: `a` (0) goes to 19 = `t`, `b` (1) goes to 26 mod 26 = 0 = `a`, `c` (2) goes to 33 mod 26 = 7 = `h`, and so on for all 26 letters.

Nothing in the attack relied on this — the letters were recovered one at a time from frequencies and words, which is what the task asked for. But it is a strong confirmation that the solution is correct: a key that was even slightly wrong would show no such structure, so the odds of stumbling onto a clean arithmetic rule by accident are negligible.

The recovered message

```
THE TELEGRAPH made cryptography what it is today. Samuel F. B. Morsesent "What hath God wrought!" in
1844. The next year his lawyer andpromotional agent, Francis O. J. Smith, published a commercial
codeentitled The Secret Corresponding Vocabulary; Adapted for Use to Morse'sElectro-Magnetic Telegraph,
in whose preface he declared that "secrecy incorrespondence, is far t he most important consideration."
This wasprovided by a super - encipherment.As the most exciting invention of the first half of the
century, thetelegraph stirred as much interest in its day as Sputnik did in its. Thegreat and widely
felt need for secrecy aw akened the latent interest inciphers that so many people seem to have, and
kindled a new interest inmany others. Dozens of persons attempted to dream up their ownunbreakable
ciphers. Their contributions enriched it with dozens of newcipher systems.As busin essmen and the
public
used the telegraph more and more,they found that their fears about lack of privacy were exaggerated.
Theclerks dealt impersonally with the messages. The telegraph companiesrespected their confidentiality.
And commercial codes like Smith's, whichreplaced words and phrases by single codewords or code –numbers
to cut telegraph tolls, afforded sufficient security for mostbusiness transactions by simply precluding
```

```
an at-sight comprehension ofthe meaning. The brokers and traders soon realized that the mainadvantage
of
these codes was their economy.Government ministries used the telegraph, too. At first they musthave
encoded with their nomenclators. But although secrecy wasparamount for them, they liked the telegraphic
economy of a large code —especially as they telegraphed more and more. So when the time arrivedto
compile a new nomenclator, they abandoned that form, copied thecommercial form, and produced a full
-fledged code. The nomenclatorshad had their 1,- or 2,000 code-numbers in mixed order , but the warand
foreign ministries balked at the expense of drawing up a 50,000 -entry code in two parts, and they had
no professional cryptanalysts towarn them of the danger of the one -part format. They relied for
securityupon small editions, big safes, e xtensive lexicon (large codes are harderto break than small
ones, other things being equal), andsuperencipherment, retaining codenumbers to facilitate this instead
ofswitching to codewords. This evolution was essentially complete by the1860s. The large, one-part code
had replaced the small, two -partnomenclator in high-level military and diplomatic
cryptography.Meanwhile, the telegraph, author of this development, was creatingsomething new in
war-signal communications, or voluminous commandand reconnaissance messages. Of course such messages
had
existedbefore, with torches, pigeons, and couriers, but in so rarefied a form thatthey were not even
called "signal communications." The telegraphenabled commanders, for the first time in history, to
exert
instantaneo usand continuous control over great masses of men spread over largeareas.These tactical
messages required protection: telegraph wires could betapped. Neither the old nomenclator nor the new
code would do. Theywere too easy to capture in combat, too hard to reissue quickly andfrequently to the
numerous and widespread telegraph posts. Signalofficers turned away from them. They looked instead to
that neglectedchild of cryptography, the cipher. Ciphers could be printed cheaply on asingle sheet of
paper and dist ributed with ease. Secrecy was based uponvariable keys, so capture of the general system
and even of one of thekeys would not compromise all an army's secret messages. Solutionswould be
prevented by rapid key changes. Ciphers were ideal for battle -zone messages, and the first of the
modern wars, the American Civil War,used them for just that. Thus was born a new genre in cryptography:
thefield cipher.
```

The words that run together (`Morsesent`, `thetelegraph`) and the stray spaces inside words (`busin essmen`, `e xtensive`) are inherited from the cryptogram as it was given, not introduced by the decryption — every one of them is present in the ciphertext in the same place.

The passage is an extract on the effect of the telegraph on nineteenth-century cryptography, from David Kahn's history *The Codebreakers*.

6. Implementation

The analysis was done with a small Python 3 program, written for this laboratory work, which uses only the standard library.

File	Contents
`english.py`	English letter frequencies (table 2.2) and the language model
`frequency.py`	letter, bigram, trigram and word statistics
`substitution.py`	applying a full or partial key to a text
`walkthrough.py`	the eight deductions above, as data
`solver.py`	automatic key recovery by hill climbing
`main.py`	the command-line program

```
python3 main.py analyse      # the frequency tables
python3 main.py walkthrough  # the attack, one deduction at a time
python3 main.py solve        # recover the key automatically
python3 main.py decrypt      # the plaintext
```

Showing a half-broken message

```
def partial_decrypt(text, key):
    out = []
    for char in text:
        if not char.isalpha():
            out.append(char)
            continue
        lowered = char.lower()
        if lowered in key:
            out.append(key[lowered])
        else:
            out.append(lowered.upper())
    return "".join(out)
```

This is what produces the intermediate texts in section 4. A letter already recovered is written in lowercase, a letter still unknown is left as the uppercase ciphertext letter, and anything that is not a letter is passed through untouched — which is what keeps the word boundaries and punctuation visible while the message is only half broken.

Repetition patterns

```
def pattern(word):
    seen = {}
    out = []
    for letter in word:
        if letter not in seen:
            seen[letter] = ALPHABET[len(seen)].upper()
        out.append(seen[letter])
    return "".join(out)
```

A substitution cannot change the shape of a word, so `pattern("wqvw")` and `pattern("that")` are both `ABCA`. Comparing patterns is what narrows a cipher word down to a handful of candidates, and it is the reason `VYUVGPV` could only be `EXPENSE`.

Recovering the key automatically

The manual attack was also checked by a program that knows nothing about the message. It starts from the key that matches the two frequency orders — the opening move of the manual attack — and then improves it by swapping pairs of letters:

```
def _climb(key, scorer, rng):
    """Swap plaintext letters until no single swap improves the score."""
    best = dict(key)
    best_score = scorer.score(best)
    letters = list(ALPHABET)
    improved = True
    while improved:
        improved = False
        pairs = [(a, b) for i, a in enumerate(letters) for b in letters[i + 1:]]
        rng.shuffle(pairs)
        for first, second in pairs:
            candidate = dict(best)
            candidate[first], candidate[second] = best[second], best[first]
            candidate_score = scorer.score(candidate)
            if candidate_score > best_score:
                best, best_score = candidate, candidate_score
                improved = True
    return best, best_score
```

The score decides what counts as progress, and it combines two signals because neither is enough alone:

```
for word in self.words:
    plain = word.translate(table)
    padded = "_" + plain + "_"
    for index in range(len(padded) - 1):
        bigram_total += self.bigrams.get(padded[index:index + 2], self.floor)
    if plain in self.dictionary:
        word_total += len(plain) ** 2
```

Letter-pair statistics give a usable gradient while the text is still gibberish, when no real words have appeared yet; counting words of the dictionary sharpens the last steps, weighted by the square of the word length because a long word matching is far stronger evidence than a short one. Hill climbing can still stall in a local maximum, so the search restarts from shuffled keys and keeps the best result.

Run on this cryptogram the program returns `bqfujyncrgvkzodshwlapetixm` — the same key recovered by hand, and the main confirmation that the manual attack did not go wrong anywhere.

Checks

Sixteen automated checks cover the pieces: that the key is a one-to-one mapping of all 26 letters, that only letters are substituted and capitalisation is preserved, that every step of the walkthrough only adds letters and that the eight steps end at the recovered key, and that the plaintext contains the phrases it should. Two of them are worth singling out:

- the solver is given the recovered plaintext re-enciphered with a *random* key and has to break that too, which tests the method rather than the answer;
- the key is checked against the affine rule `c = (7x + 19) mod 26` for all 26 letters.

```
$ python3 -m unittest discover -s tests
................
----------------------------------------------------------------------
Ran 16 tests in 8.408s

OK
```

7. Conclusions

The cipher has $26! \approx 4.03 \times 10^{26}$ possible keys and was broken in eight steps, by hand, without trying a single one of them. That is the whole lesson of the laboratory work: the size of the key space says almost nothing about the strength of a cipher. What matters is whether the ciphertext still carries the structure of the language, and a monoalphabetic substitution carries nearly all of it.

The attack was not really a search. Each step was a hypothesis that could be read back and checked immediately — if `WQV` had not been `THE`, the next line would have been nonsense and the guess abandoned at no cost. Because the evidence accumulates, the work gets easier as it goes: the first deduction needed the frequency table, and the last seven letters fell out of single words in which they were the only unknown.

Two details were worth noticing in practice. First, the frequency table is a way in, not an answer: lining up the two orders rank for rank was correct for only 9 of the 26 letters. It was correct for `E`, `T`, `A` and `O`, and that was enough to make the text readable; everything after that came from words, not counts. Second, what gave the message away fastest was not the letters at all but what was left *around* them — the word boundaries, the punctuation and the capitalisation. `WQV` standing alone 35 times is a much

louder signal than any frequency, and a cipher that stripped the spaces would have been appreciably harder work.

The practical conclusion is that a monoalphabetic cipher is unusable for anything real. Hiding the word boundaries, padding the text or enciphering in fixed-length groups would slow an attacker down but would not fix the underlying flaw, since one plaintext letter still always produces the same ciphertext letter. The flaw is only repaired by making the substitution change as the message goes on — which is what polyalphabetic ciphers do, and which is the next thing to study.

8. Source code

The full source code is in the repository github.com/alo42/CS, in the `LAB2` directory.

```
git clone git@github.com:alo42/CS.git
cd CS/LAB2
python3 main.py walkthrough
```

`solve` additionally needs an English word list; it uses `/usr/share/dict/words`, present on macOS and most Linux systems, and the environment variable `ENGLISH_WORD_LIST` overrides the path. Everything else needs only Python 3 and the standard library.